Claims Processor — Project Submission

Project overview

Claims Processor is a human-in-the-loop application that turns claim documents into editable, traceable JSON records. It focuses on CMS-1450 institutional claims and also recognizes CMS-1500 professional claims. The goal is to reduce manual transcription while keeping reviewers responsible for verifying extracted information.

User workflow

The reviewer uploads a PDF or image on **Review by HITL**, then presses **Analyzer** to start OCR. A split view displays the original document alongside fields organized into logical groups. Source highlights help reviewers verify values, correct errors and review service charges with separate dollar/cent inputs. Saving creates one library record per claim, with options to reopen, view and download its JSON. The JSON preserves all defined fields, including blanks, original values, saved revisions and the edit history.

Technical approach

The application combines a React/TypeScript frontend with a Python FastAPI backend. RapidOCR performs local recognition, with image preprocessing and form-specific field mapping. LangGraph coordinates analysis, review and indexing. SQLite stores records and audit history, while JSON files mirror saved claims. Optional GPT integration generates schema definitions from a value-free field catalog; actual schema validation runs locally. Without OpenAI credentials, the application remains usable with a clearly identified local schema. The source and run configuration are ready for Replit deployment.

RAG comparison

The project also implements claim question answering using LangChain interfaces, local vector search and an optional Pinecone adapter. Retrieval Insights compares fixed-size and logical semantic chunking, each with and without reranking. The controlled benchmark uses 45 held-out questions across nine synthetic claims and reports evidence recall, ranking quality, lookup accuracy and latency, with downloadable chunking and reranking analyses. These experiments assess retrieval rather than OCR accuracy or insurance coverage decisions.

Validation and limitations

Recorded local verification includes 37 passing backend tests, browser checks for review/save/export and mobile workflows, TypeScript validation and a production frontend build. Authored files remain below 100 lines. The prototype demonstrates document analysis, human correction, auditable JSON storage and retrieval evaluation. Live GPT and Pinecone connections were not verified without credentials; Replit hosting has not been deployed. Handwriting accuracy, complete UB-04 subfield/payer-rule coverage and production security/compliance require further work. Benchmark labels are template-authored and provisional, so synthetic results do not establish production accuracy.